✅ **I checked your app at** https://fmc.reidar.tech

The site is a solid, functional React-based Boeing 737 NG CDU trainer. It already has good structure, many function keys, LSK support (via arrows), and a clean monospaced display. However, from both the live site and our earlier analysis, the **visuals** are still the biggest area for improvement compared to real hardware.

Current Visual State (Summary)

- Uses a **single-color** text scheme (mostly one bright/light color on dark background) — real 737 NG CDUs are multi-color.
- Monospaced font that works but lacks the specific character spacing and weight of real FMC hardware.
- Function keys are present and mostly complete, but button styling and feedback feel digital rather than physical.
- LSKs use generic arrows (◄ / ►) instead of contextual labels.
- Placeholders and cursor behavior are basic.
- Overall look is clean and usable, but it reads more like a **good simulation** than a close replica of real cockpit hardware.

Prioritized Steps to Enhance the Visuals

Here's a practical, phased approach aligned with the accuracy improvement plan (especially **Phase 1**). I've ranked them by **impact vs effort**.

1. Multi-Color Display Support (Highest Impact – Do This First)

Current problem: Everything is one color. Real NG CDUs use specific colors for meaning.

Steps:

- Create a color token system in Tailwind/config:
  - `cyan` for inactive titles
  - `green` for VOR/navaid data
  - `magenta` for active guidance
  - `white` for normal data
  - Shaded/inverse white for modified data (when EXEC is lit)
- Update all page renderers (`shared/src/fmc/pages/`) to return `DisplayLine` objects with a `color` property.
- Modify the CDU Display component to render text with the correct Tailwind color classes (or inline styles) based on the color token.
- Add support for “modified” styling (lighter shade or subtle background).

Expected result: The display will immediately look much more like a real 737 NG CDU.

2. Improve Font & Character Rendering

Real CDUs have very specific character spacing and weight.

Steps:

- Replace or supplement the current monospace font with a better FMC-style font (options: tuned monospace + letter-spacing, or a bitmap-style font rendered via canvas for pixel-perfect control).
- Add slight negative `letter-spacing` and adjust `line-height` to match real CDU character cells.
- Consider rendering the display using a `<canvas>` element for the highest fidelity (many high-end trainers do this).

3. Add Missing / Refine Function Keys

From the live site, many keys exist, but visual consistency and completeness matter.

Steps:

- Ensure **all 14 function keys** are clearly visible and correctly labeled:
  - Top row: `INIT REF`, `RTE`, `CLB`, `CRZ`, `DES`
  - Second row: `DIR INTC`, `LEGS`, `DEP ARR`, `HOLD`, `PROG`, `N1 LIMIT`, `FIX`
  - Navigation: `PREV PAGE`, `NEXT PAGE`
- Improve button styling: Add subtle gradients, borders, and better hover/active states to make them feel more physical.
- Add a small “backlight” glow effect on active/pressed keys.

4. Better Button Feedback & Tactile Feel

Steps:

- Add smooth press animation (scale + shadow change on click/touch).
- Implement different visual states: `normal`, `pressed`, `highlighted`, `disabled`.
- Add subtle key “wobble” or depth effect on press (can be done with CSS transforms + transition).
- For LSKs: Show contextual text labels when space allows (e.g., “SET ORIGIN”) instead of just arrows.

5. Context-Sensitive Placeholders & Cursor

Steps:

- Implement smart placeholders:
  - Alphabetic fields → `----`
  - Numeric fields → `....`
  - Decimal fields → `--.-`
  - Flight levels → `FL---`
- Add a blinking underscore cursor on the active input field (scratchpad or line).

6. Overall Physical Realism (Bezel, Spacing, Polish)

Steps:

- Improve the outer bezel with better shadows, slight bevel, and realistic aluminum-like styling.
- Tighten spacing between keys and the display to better match real hardware proportions.
- Add subtle screen glow or very light scanline effect (keep it minimal to maintain performance).
- Ensure excellent touch targets (already good) and add ripple feedback on mobile/iPad.

7. Performance & Technical Considerations

While improving visuals:

- Keep **≥ 55 fps** on iPad (test with React DevTools Profiler).
- Use `React.memo` and proper keys on display lines.
- If using canvas for the display, optimize redraws heavily.
- Run visual regression tests (Playwright) against real CDU reference photos after major changes.

Recommended Order of Work (Phase 1 Focus)

Priority	Task	Impact	Effort	Suggested Order
1	Multi-color display support	Very High	Medium	Do first
2	Better font + character spacing	High	Medium	Do early
3	Complete/refine 14 function keys	High	Medium	Parallel with #1
4	Button press feedback & states	Medium	Low	After colors
5	Contextual LSK labels + placeholders	Medium	Low	After colors
6	Bezel & physical polish	Medium	Medium	Later in phase

Quick Wins You Can Do Today

1. Change the display text color from single bright green/white to a proper multi-color system (start with cyan titles + magenta guidance).
2. Add the few missing function keys if any are absent.
3. Improve button hover and active states.

Would you like me to:

- Give you **specific code suggestions** for implementing multi-color display or better font rendering?
- Help create a **color token system** + updated `DisplayLine` type?
- Refine the accuracy plan further with these visual steps as explicit tasks?

Just tell me where you want to go next. The foundation is already good — these visual upgrades will make a dramatic difference.